PYTHON CODE

```
import os
print(rf"C:\Users\{os.getlogin()}\pulse\data\aggregated\transaction\country\india\state\\")


import os
import json
import pandas as pd


# 1. Update this to your LOCAL 'aggregated/user' state folder
path = r"C:\Users\kartik\Downloads\pulse-master\pulse-
master\data\aggregated\user\country\india\state"


user_data = []


if not os.path.exists(path):
    print("Path not found. Please verify the folder exists at:", path)
else:
    for state in os.listdir(path):
        p_state = os.path.join(path, state)
        if os.path.isdir(p_state):
            for year in os.listdir(p_state):
                p_year = os.path.join(p_state, year)
                if os.path.isdir(p_year):
                    for file in os.listdir(p_year):
                        p_file = os.path.join(p_year, file)
                        with open(p_file, 'r') as f:
                            d = json.load(f)


                            # Core metrics
                            reg_users = d['data']['aggregated']['registeredUsers']
```

```
            app_opens = d['data']['aggregated']['appOpens']

            # Device breakdown logic
            # Note: Sometimes usersByDevice is None in the dataset
            if d['data']['usersByDevice'] is not None:
              for device in d['data']['usersByDevice']:
                user_data.append({
                  "State": state,
                  "Year": year,
                  "Quarter": int(file.strip('.json')),
                  "Registered_Users": reg_users,
                  "App_Opens": app_opens,
                  "Brand": device['brand'],
                  "Count": device['count'],
                  "Percentage": device['percentage']
                })

  # 2. Convert and Save
  df_user = pd.DataFrame(user_data)
  df_user.to_csv("phonepe_users.csv", index=False)
  print(f"Success! Saved {len(df_user)} rows to 'phonepe_users.csv'")

#MAP

import os
import json
import pandas as pd

# CHANGE THIS PATH for each category (Map or Top)
path = r"C:\Users\kartik\Downloads\pulse-master\pulse-master\data\map\transaction\hover\country\india\state"
```

```
map_data = []

for state in os.listdir(path):
    p_state = os.path.join(path, state)
    if os.path.isdir(p_state):
        for year in os.listdir(p_state):
            p_year = os.path.join(p_state, year)
            for file in os.listdir(p_year):
                p_file = os.path.join(p_year, file)
                with open(p_file, 'r') as f:
                    d = json.load(f)
                    for i in d['data']['hoverDataList']:
                        map_data.append({
                            "State": state,
                            "Year": year,
                            "Quarter": int(file.strip('.json')),
                            "District": i['name'],
                            "Count": i['metric'][0]['count'],
                            "Amount": i['metric'][0]['amount']
                        })

df_map = pd.DataFrame(map_data)
df_map.to_csv("map_transactions.csv", index=False)
print("Saved map_transactions.csv")

# 'top' folder
path = r"C:\Users\kartik\Downloads\pulse-master\pulse-master\data\top\transaction\country\india\state"

top_data = []
```

```
for state in os.listdir(path):
    p_state = os.path.join(path, state)
    if os.path.isdir(p_state):
        for year in os.listdir(p_state):
            p_year = os.path.join(p_state, year)
            for file in os.listdir(p_year):
                p_file = os.path.join(p_year, file)
                with open(p_file, 'r') as f:
                    d = json.load(f)
                    for i in d['data']['pincodes']:
                        top_data.append({
                            "State": state,
                            "Year": year,
                            "Quarter": int(file.strip('.json')),
                            "Pincode": i['entityName'],
                            "Count": i['metric']['count'],
                            "Amount": i['metric']['amount']
                        })

df_top = pd.DataFrame(top_data)
df_top.to_csv("top_transactions.csv", index=False)
print("Saved top_transactions.csv")

import pandas as pd
from sqlalchemy import create_engine

# --- CONFIGURATION ---
# Replace 'root' with your MySQL username
# Replace 'your_password' with your MySQL password
# Replace 'phonepe_db' with your actual database name
```

```
user = "root"
password = "1234"
host = "localhost"
database = "phonepe_db"

# Create the connection engine
engine = create_engine(f"mysql+pymysql://{user}:{password}@{host}/{database}")

def upload_to_sql(file_name, table_name):
    try:
        # Read the CSV you created earlier
        df = pd.read_csv(file_name)

        # Push to SQL (if_exists='replace' creates the table for you)
        df.to_sql(table_name, engine, if_exists='replace', index=False)
        print(f"✅ Successfully uploaded {file_name} to table '{table_name}'")
    except Exception as e:
        print(f"❌ Error uploading {file_name}: {e}")

# --- EXECUTION ---
# List of all the CSVs you have created
files_to_upload = {
    "phonepe_transactions.csv": "aggregated_transaction",
    "phonepe_users.csv": "aggregated_user",
    "map_transactions.csv": "map_transaction",
    "top_transactions.csv": "top_transaction"
}

for csv_file, table_name in files_to_upload.items():
    upload_to_sql(csv_file, table_name)
```

```
print("\nAll data is now in your SQL database!")


import pymysql


try:
    conn = pymysql.connect(
        host='localhost',
        user='root',
        password='12345', # Change this
        database='phonepe_db'          # Change this
    )
    print(" ✅ Connection Successful!")
    conn.close()
except Exception as e:
    print(f" ❌ Connection Failed: {e}")


import pandas as pd
from sqlalchemy import create_engine


# --- CONFIGURATION ---
user = "root"
password = "12345"
host = "localhost"
database = "phonepe_db"


# Create the engine using the verified credentials
engine = create_engine(f"mysql+pymysql://{user}:{password}@{host}/{database}")


def upload_to_sql(file_name, table_name):
    try:
        df = pd.read_csv(file_name)
```

```
        # 'replace' will create the table if it doesn't exist
        df.to_sql(table_name, engine, if_exists='replace', index=False)
        print(f"✅ Successfully uploaded {file_name} to table '{table_name}'")
    except Exception as e:
        print(f"❌ Error uploading {file_name}: {e}")

# --- EXECUTION ---
files_to_upload = {
    "phonepe_transactions.csv": "aggregated_transaction",
    "phonepe_users.csv": "aggregated_user",
    "map_transactions.csv": "map_transaction",
    "top_transactions.csv": "top_transaction"
}

for csv_file, table_name in files_to_upload.items():
    upload_to_sql(csv_file, table_name)

print("\n🚀 All data is now live in MySQL!")
```

DATASETS

map_transactions.csv

top_transactions.csv

phonepe_users.csv

phonepe_transactions.csv

SQL QUERY

```
CREATE DATABASE IF NOT EXISTS phonepe_db;


use phonepe_db;


USE phonepe_db;

SELECT * FROM aggregated_transaction LIMIT 10;
```

STREAMLIT CODE

```
import streamlit as st
import pandas as pd
import plotly.express as px

# --- 1. SETTINGS & STYLING ---
st.set_page_config(page_title="PhonePe Pulse PRO", layout="wide", page_icon="📈")

# The fix for your previous error is 'unsafe_allow_html=True'
st.markdown("""
  <style>
  .main { background-color: #0e1117; }
  [data-testid="stMetricValue"] { font-size: 28px; color: #6e42ff; }
  .stMetric {
    background-color: #1e2130;
    padding: 20px;
    border-radius: 12px;
    border: 1px solid #3e4259;
  }
  </style>
  """, unsafe_allow_html=True)

# --- 2. DATA LOADERS ---
@st.cache_data
def load_and_clean_data():
  try:
    t_df = pd.read_csv("phonepe_transactions.csv")
    u_df = pd.read_csv("phonepe_users.csv")
    m_df = pd.read_csv("map_transactions.csv")
```

```
    # Professional Cleaning: Sync state names for the India Map
    for df in [t_df, u_df, m_df]:
        df['State'] = df['State'].str.replace('-', ' ').str.title()
        df['State'] = df['State'].str.replace('Andaman & Nicobar Islands', 'Andaman & Nicobar')
        df['State'] = df['State'].str.replace('Dadra & Nagar Haveli & Daman & Diu', 'Dadra and Nagar Haveli and Daman and Diu')
    return t_df, u_df, m_df
  except Exception as e:
    return None, None, str(e)

trans_df, user_df, map_df = load_and_clean_data()

# Error handling if CSVs are missing
if trans_df is None:
  st.error(f"⚠️ CSV files not found or Error: {map_df}. Please ensure your CSV files are in the main folder of your GitHub repo.")
  st.stop()

# --- 3. SIDEBAR FILTERS ---
with st.sidebar:
  st.title("📊 Control Panel")
  year = st.selectbox("Select Year", sorted(trans_df['Year'].unique(), reverse=True))
  quarter = st.select_slider("Select Quarter", options=[1, 2, 3, 4])
  st.divider()
  st.info("This dashboard visualizes PhonePe's transaction and user data across India.")

# Apply filters
f_trans = trans_df[(trans_df['Year'] == year) & (trans_df['Quarter'] == quarter)]
f_map = map_df[(map_df['Year'] == year) & (map_df['Quarter'] == quarter)]
f_user = user_df[(user_df['Year'] == year) & (user_df['Quarter'] == quarter)]
```

```
# --- 4. DASHBOARD HEADER ---
st.title("📱 PhonePe Pulse Analytics")
st.caption(f"Showing insights for Financial Year {year} | Quarter {quarter}")

# Top KPI Metrics
m1, m2, m3 = st.columns(3)
total_val = f_trans['Transaction_Amount'].sum()
total_cnt = f_trans['Transaction_Count'].sum()
total_reg = f_user['Registered_Users'].sum()

m1.metric("Total Transaction Value", f"₹{total_val/1e7:.2f} Cr")
m2.metric("Total Transactions", f"{total_cnt/1e5:.2f} L")
m3.metric("Registered Users", f"{total_reg/1e5:.2f} L")

st.divider()

# --- 5. VISUALIZATIONS ---
row1_col1, row1_col2 = st.columns([1.3, 0.7])

with row1_col1:
  st.subheader("🗺️ India Transaction Heatmap")
  # Group by state for map
  map_data = f_map.groupby("State")["Amount"].sum().reset_index()

  fig_india = px.choropleth(
    map_data,
geojson="https://gist.githubusercontent.com/jbrobst/56c13bbbf9ad97d3121354efebd9e57e/raw/0ad293846248c1d3008a30ad47e440590f1a3229/india_states.geojson",
    featureidkey='properties.ST_NM',
```

```
    locations='State',
    color='Amount',
    color_continuous_scale="Viridis",
    template="plotly_dark",
    hover_name="State"
  )
  fig_india.update_geos(fitbounds="locations", visible=False)
  fig_india.update_layout(margin={"r":0,"t":0,"l":0,"b":0}, height=500)
  st.plotly_chart(fig_india, key="india_map")

with row1_col2:
  st.subheader(" 📊 Category Breakdown")
  type_data = f_trans.groupby("Transaction_Type")["Transaction_Amount"].sum().reset_index()
  fig_pie = px.pie(type_data, values='Transaction_Amount', names='Transaction_Type',
hole=0.5)
  fig_pie.update_layout(showlegend=False, height=450)
  st.plotly_chart(fig_pie, key="type_pie")

# --- 6. TOP PERFORMERS ---
st.divider()
st.subheader(f" 🏆 Top 10 States by Transaction Volume")
top_states =
f_trans.groupby("State")["Transaction_Amount"].sum().sort_values(ascending=False).head(10).
reset_index()

fig_bar = px.bar(top_states, x="Transaction_Amount", y="State", orientation='h',
          color="Transaction_Amount", color_continuous_scale="Purples")
fig_bar.update_layout(yaxis={'categoryorder':'total ascending'})
st.plotly_chart(fig_bar, key="top_states_bar")

st.caption("Data source: PhonePe Pulse Open Data | Developed by Kartik")
```